

JavaScript Access and Properties

1. Write program using this keyword when we click on button, it should be disappear.

Answer:

```
<!DOCTYPE html>
<html>
<head>
  <title>this Keyword</title>
</head>

<body>
<button onclick="hideButton(this) ">
  Click Me
</button>
<script>
function hideButton(btn) {
  btn.style.display = "none";
}
</script>
</body>
</html>
```

Output:

When the button is clicked, it disappears.

2. Invoke a function using call() and apply(). Also Spot the difference between them.

Answer:

```
<!DOCTYPE html>
<html>
<head>
  <title>Call and Apply</title>
</head>

<body>
<script>
var person = {
  fullname: function(city, country){
    return this.firstName + " " + this.lastName +
      " from " + city + ", " + country;
  }
};

var student = {
  firstName: "Suman",
  lastName: "Kumar"
};

document.write(person.fullname.call(student, "Bangalore", "India"));
document.write("<br><br>");

document.write(person.fullname.apply(student, ["Mangalore", "India"]));

</script>
</body>
</html>
```

Difference between `call()` and `apply()` :

call()	apply()
Arguments are passed individually	Arguments are passed as an array
Syntax: <code>call(obj, arg1, arg2)</code>	Syntax: <code>apply(obj, [arg1, arg2])</code>
Suitable for known arguments	Suitable when arguments are already in an array

3. Write a program for COUNTER. Whenever we click on a "Count" button, value should increment.

Answer:

```
<!DOCTYPE html>
<html>
<head>
  <title>Counter</title>
</head>

<body>
<h2 id="count">0</h2>
<button onclick="increment()">
Count
</button>
<script>
let count = 0;
function increment(){
count++;
document.getElementById("count").innerHTML = count;
}

</script>
</body>
</html>
```

4. Create a Counter with the help of getter and setter accessors.

Answer:

```
<!DOCTYPE html>
<html>
<head>
  <title>Getter and Setter</title>
</head>

<body>

<script>

let counter = {

value: 0,

get count(){
  return this.value;
```

```

},

set count(num) {
    this.value = num;
}

};

counter.count = 10;

document.write("Counter Value : " + counter.count);

</script>

</body>
</html>

```

5. Add some more properties to an existing function using object prototypes.

Answer:

```

<!DOCTYPE html>
<html>
<head>
    <title>Prototype Example</title>
</head>

<body>

<script>

function Person(name) {
    this.name = name;
}

Person.prototype.city = "Bangalore";

Person.prototype.display = function() {
    return this.name + " - " + this.city;
};

let p1 = new Person("Suman");

document.write(p1.display());

</script>

</body>
</html>

```

Things to know:

1. Know keywords and existing functions of JavaScript?

Answer:

Common JavaScript Keywords:

Keyword	Purpose
var	Declares a variable (function-scoped)
let	Declares a block-scoped variable
const	Declares a constant value
function	Declares a function
return	Returns a value from a function
if	Conditional statement
else	Alternative block for if
for	Loop statement
while	Repeats a block while a condition is true
switch	Multi-way conditional statement
break	Exits a loop or switch
continue	Skips the current iteration of a loop
this	Refers to the current object
new	Creates an object from a constructor
class	Declares a class
try	Starts exception handling
catch	Handles exceptions
throw	Throws an exception

Common Built-in JavaScript Functions:

Function	Purpose
alert()	Displays an alert box
prompt()	Accepts input from the user
confirm()	Displays a confirmation dialog
parseInt()	Converts a string to an integer
parseFloat()	Converts a string to a floating-point number
isNaN()	Checks whether a value is Not-a-Number
setTimeout()	Executes code after a specified delay
setInterval()	Executes code repeatedly at a specified interval
clearTimeout()	Cancels a timeout
clearInterval()	Cancels an interval
console.log()	Prints messages to the browser console
document.getElementById()	Selects an element by its ID
document.querySelector()	Selects the first matching element using a CSS selector